CC3067 REDES · THE INTERNET OF THINGS

Estación meteorológica IoT

Sensores simulados que publican telemetría a un broker Apache Kafka en el Edge, consumo con gráfica en vivo y codificación en un payload de 3 bytes.

INTEGRANTES

Hugo Barillas · 23306

Luis Palacios · 23933

TOPIC KAFKA

23306

Universidad del Valle de Guatemala

Noviembre 2026

ARQUITECTURA

Del sensor a la gráfica, pasando por el Edge

PRODUCER · “LA COSA”

Nodo de sensores

Simula termómetro, higrómetro y veleta. Publica una lectura cada 15–30 s hasta ser interrumpido.

`producer.py · sensor.py`

EDGE SERVER

Broker Apache Kafka

161.35.11.141:9092
SASL_PLAINTEXT · PLAIN
Topic **23306**, key “sensor1”

Persiste los eventos por partición y offset.

CONSUMER

Dashboard en vivo

Se suscribe al topic, decodifica cada mensaje y redibuja temperatura, humedad y viento.

`consumer.py · matplotlib`

Patrón **Publish/Subscribe**: el sensor no conoce al consumidor; ambos solo conocen el broker y el topic. El mismo código soporta dos formatos de payload: **JSON** (fases 3.2–3.3) y **compacto de 3 bytes** (fase 3.4).

FASE 3.1 · SIMULACIÓN DE UN SENSOR

Tres sensores, lecturas centradas en 55

Sensor	Rango y tipo	Generación
Temperatura	0–110.00 °C, float 2 dec.	Normal μ=55, σ=15, acotada
Humedad relativa	0–100 %, entero	Normal μ=55, σ=15, redondeada
Dirección del viento	N, NO, O, SO, S, SE, E, NE	Uniforme entre las 8

Con σ = 15, el 99.7 % de las muestras cae entre 10 y 100, así que el recorte a los límites casi nunca se activa y la gráfica se ve realista. Cada Reading valida sus rangos al crearse.

sensor.py

```
temperatura = round(clamp(
  rng.gauss(55, 15), 0, 110), 2)
humedad = int(round(clamp(
  rng.gauss(55, 15), 0, 100)))
direccion = rng.choice(WIND_DIRECTIONS)
```

Payload JSON (57-58 bytes)

```
{"temperatura":56.32,"humedad":51,
 "direccion_viento":"SO"}
```

PREGUNTAS · FASE 3.1

JSON en el modelo OSI y sus beneficios

¿A qué capa pertenece JSON/SOAP según el modelo OSI y por qué?

A la **capa 6, Presentación**. Esa capa define cómo se representan, serializan y codifican los datos para que dos sistemas distintos los interpreten igual, sin importar lenguaje o arquitectura. JSON es justamente un formato de representación de datos.

SOAP, además de su formato XML, define reglas de intercambio de mensajes, por lo que también roza la capa 7 (Aplicación). En el modelo TCP/IP ambos quedan en la capa de Aplicación.

¿Qué beneficios tiene utilizar un formato como JSON/SOAP?

- **Interoperabilidad:** independiente del lenguaje; Python produce y cualquier cliente consume.
- **Autodescriptivo y legible:** cada campo tiene nombre, fácil de depurar.
- **Extensible:** agregar un campo no rompe a los consumidores existentes.
- **Desacopla el código** del orden de bytes; menos frágil.
- **Validable** con esquemas (JSON Schema, XSD).

Costo: es verboso. Nuestra lectura ocupa ~58 bytes en JSON y 3 bytes en binario.

FASE 3.2 · ENVÍO DE DATOS AL SERVER EDGE

El dispositivo IoT publica cada 15–30 s

CONEXIÓN

bootstrap 161.35.11.141:9092, SASL_PLAINTEXT / PLAIN. Credenciales en .env, fuera del repositorio.

CONFIABILIDAD

acks=all + idempotencia; un callback confirma partición y offset de cada mensaje.

CICLO DE VIDA

Bucle hasta Ctrl+C; espera aleatoria de 15–30 s; al salir hace flush() sin perder mensajes.

```
$ python -m weather_station.producer · log real, 10 nov 2026
20:46:56 INFO Publicando en topic '23306' (modo json). Ctrl+C para detener.
20:46:56 INFO Lectura {'temperatura': 41.89, 'humedad': 69, 'direccion_viento': 'NO'} → payload
b'{"temperatura":41.89,"humedad":69,"direccion_viento":"NO"}'
20:47:25 INFO Deteniendo: enviando mensajes pendientes...
20:47:25 INFO Entregado a 23306 [particion 2, offset 3] (58 bytes)
```

FASE 3.3 · CONSUMIR Y DESPLEGAR

Un consumer que grafica cada mensaje que llega

SUSCRIPCIÓN

Group propio con earliest: carga la telemetría pasada y luego la entrante.

DECODIFICACIÓN

Por tamaño: 3 bytes → compacto; otro → JSON. Inválidos se ignoran.

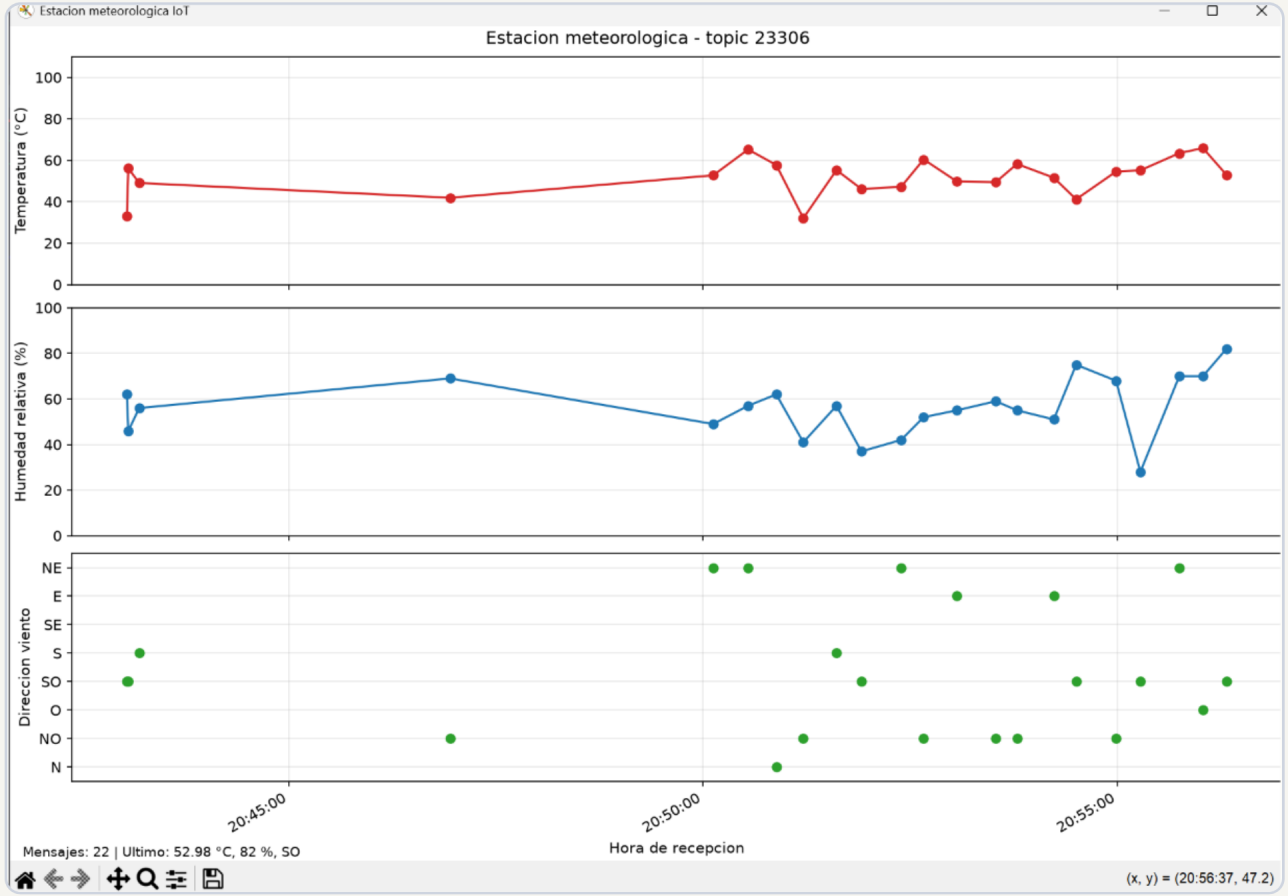
GRÁFICA EN VIVO

3 subgráficas matplotlib que se redibujan con cada mensaje.

```
$ python -m weather_station.consumer --save output/grafica.png · log real, 10 nov 2026
20:47:11 INFO Suscrito a '23306'. Cierre la ventana o Ctrl+C para salir.
20:47:12 INFO offset 0 key=b'sensor1' 58 bytes → {'temperatura': 33.06, 'humedad': 62, 'direccion_viento': 'SO'}
20:47:12 INFO offset 1 key=b'sensor1' 58 bytes → {'temperatura': 56.06, 'humedad': 46, 'direccion_viento': 'SO'}
20:47:13 INFO offset 2 key=b'sensor1' 3 bytes → {'temperatura': 49.04, 'humedad': 56, 'direccion_viento': 'S'}
20:47:13 INFO offset 3 key=b'sensor1' 58 bytes → {'temperatura': 41.89, 'humedad': 69, 'direccion_viento': 'NO'}
```

FASE 3.3 · RESULTADO

22 lecturas graficadas en tiempo real



QUÉ SE VE

Temperatura, humedad y viento vs. hora. Oscilan alrededor de 55.

RITMO

Un punto cada 15–30 s; abajo, conteo y última lectura.

DETALLE

El 3.er punto (49.04 °C, 56 %, S) llegó como payload de **3 bytes**.

Captura del 10 nov 2026, 20:56 · topic 23306

PREGUNTAS · FASE 3.3

Pub/Sub con Kafka: cuándo conviene

¿Qué ventajas y desventajas tiene el enfoque Pub/Sub de Kafka?

VENTAJAS

- Desacopla productores y consumidores.
- Escala con particiones y consumer groups.
- Persiste: permite releer el historial.
- Tolera fallos con réplicas.

DESVENTAJAS

- Infraestructura compleja de operar.
- Broker central: si cae, todo se detiene.
- Orden solo dentro de una partición.
- Cliente pesado para microcontroladores.

¿Para qué aplicaciones tiene sentido usar Kafka? ¿Para cuáles no?

SÍ TIENE SENTIDO

- Telemetría IoT agregada en Edge o core.
- Logs y métricas de alto volumen.
- Pipelines, ETL y change data capture.
- Eventos entre microservicios.

NO TIENE SENTIDO

- Request/response síncrono (HTTP, gRPC).
- Apps pequeñas con poco tráfico.
- Sensores limitados directo: mejor MQTT.
- Colas de tareas simples.

FASE 3.4 · IOT CON RESTRICCIONES

Una lectura completa en 24 bits

bits 23-10 · 14 bits	bits 9-3 · 7 bits	2-0 · 3 b
Temperatura × 100	**Humedad**	**Viento**
0 – 11000 < 2^{14} = 16384	0 – 100 < 2^{7} = 128	0 – 7

Ejemplo real: 49.04 °C · 56 % · S

```
01001100101000 0111000 100
4904           56      4 = S
bytes: 0x4C 0xA1 0xC4 → b'L\xa1\xc4'
```

57 → 3

bytes por lectura: JSON vs. formato compacto, una reducción de 95 %.

FASE 3.4 · ENCODE / DECODE

Empaquetar y desempaquetar con desplazamientos de bits

```
encode_compact(reading) → bytes
temp = round(r.temperatura * 100) # 0..11000
wind = WIND_DIRECTIONS.index(r.direccion_viento)
packed = (temp << 10) | (r.humedad << 3) | wind
return packed.to_bytes(3, "big")
```

```
decode_compact(payload) → Reading
packed = int.from_bytes(payload, "big")
temp = (packed >> 10) & 0x3FFF # 14 bits
hum  = (packed >> 3) & 0x7F # 7 bits
wind = packed & 0x07 # 3 bits
return Reading(temp / 100, hum, WIND_DIRECTIONS[wind])
```

1 · ESCALAR

La temperatura deja de ser float: se guarda como entero en centésimas.

2 · DESPLAZAR Y UNIR

Cada campo se corre a su posición y se combinan con OR en un entero de 24 bits.

3 · SERIALIZAR

3 bytes big-endian. El decode aplica las máscaras en orden inverso.

FASE 3.4 · EVIDENCIA

3 bytes enviados, recibidos y decodificados

```
$ python -m weather_station.producer --mode compact · log real, 10 nov 2026
20:43:11 INFO Lectura {'temperatura': 49.04, 'humedad': 56, 'direccion_viento': 'S'} → payload b'L\xa1\xc4'
20:43:13 INFO Entregado a 23306 [particion 2, offset 2] (3 bytes)
```

```
$ python -m weather_station.consumer · el mismo mensaje leído del broker
20:47:13 INFO offset 2 key=b'sensor1' 3 bytes → {'temperatura': 49.04, 'humedad': 56, 'direccion_viento': 'S'}
```

PRUEBAS AUTOMATIZADAS · PYTEST

13 passed: ida y vuelta exacta en extremos, 8 direcciones y 5000 lecturas.

NOTA

El servidor del curso ya no está disponible; esta evidencia es del 10 de noviembre.

PREGUNTAS · FASE 3.4

Cuando cada bit cuenta

¿Qué complejidades introduce tener un payload restringido?

- **Esquema rígido compartido:** productor y consumidor deben usar exactamente el mismo orden de bits; cambiar un campo rompe a todos.
- **Precisión y rango limitados:** lo que no cabe se pierde.
- **Sin espacio para metadatos:** id del sensor, timestamp, versión o checksum.
- **Errores silenciosos y depuración difícil:** un bit alterado es otra lectura “válida” y b'L\xa1\xc4' no se lee a simple vista.

¿Cómo podemos hacer que el valor de temperatura quepa en 14 bits?

Con **punto fijo**: multiplicar por 100 y guardar un entero. El máximo, 110.00 °C, se vuelve 11000, y $2^{14} = 16384$, así que cabe con margen.

```
49.04 × 100 = 4904 → 14 bits
4904 ÷ 100 = 49.04 al decodificar
```

No se pierde ningún decimal: la resolución sigue siendo 0.01 °C, la misma que exige el sensor.

PREGUNTAS · FASE 3.4

Si la humedad fuera float, y si apretara más

¿Qué pasaría si la humedad fuera float con un decimal? ¿Qué decisiones tomaríamos?

0.0–100.0 × 10 = 0–1000 necesita **10 bits**. Con 14 + 10 + 3 = **27 bits** ya no cabe en 24: sobran 3.

Opción	Costo
Temp. a 1 decimal: 11+10+3 = 24	Resolución 0.1 °C
Humedad entera como hoy	Pierde el decimal
Payload de 4 bytes	Si la red lo permite

Elegimos la primera: usa los 24 bits exactos.

¿Qué parámetros o herramientas de Kafka ayudarían con restricciones más fuertes?

- **Metadatos del mensaje:** la key identifica al sensor y el timestamp de Kafka da la hora, sin gastar bits del payload.
- **Compresión y batching:** compression.type=zstd, linger.ms, batch.size.
- **Schema Registry** con Avro o Protobuf: binario compacto y versionado.
- **Kafka Connect o un puente MQTT** en el Edge para dispositivos LoRa o Sigfox.
- **Kafka Streams:** agregar o filtrar en el Edge y enviar menos.

IMPLEMENTACIÓN

Buenas prácticas aplicadas

MÓDULOS

sensor, codec, producer, consumer y config separados; cada uno con una sola responsabilidad.

SECRETOS

Credenciales en .env, ignorado por git; el repo solo trae .env.example.

ENTORNO

Entorno virtual y versiones fijadas en requirements.txt.

PRUEBAS

13 pruebas con pytest para el sensor y el codec de 3 bytes.

ENTREGA CONFIABLE

acks=all, idempotencia y flush() al detener con Ctrl+C.

VALIDACIÓN

Cada lectura valida sus rangos; el consumer ignora mensajes inválidos.

`github.com/Hugo2049/Internet-of-Things---estaci-n-meteorol-gica`

CONCLUSIONES

Lo que nos llevamos

01

Desacoplar

El sensor y el dashboard nunca se conocen: el broker en el Edge persiste y reparte los eventos.

02

El formato es diseño

JSON en 57 bytes es legible; 3 bytes binarios son eficientes. El mismo pipeline soporta ambos.

03

Diseñar para restricciones

Punto fijo, metadatos de Kafka y puentes en el Edge permiten operar con redes como LoRa o Sigfox.